

APLICADO A UNA INTERFAZ GRÁFICA

PATRÓN MVC

Integrantes: Mauricio Daniel Montiel Valencia | Jafeth Godinez Matarrita | Luis
Gerardo Sanchez Alvarez



¿QUÉ ES EL PATRÓN MVC?

Ees una arquitectura de software que organiza una aplicación en tres componentes principales: el modelo, encargado de gestionar los datos y la lógica; la vista, responsable de mostrar la interfaz gráfica al usuario; y el controlador, que actúa como intermediario entre ambos, procesando las acciones del usuario y coordinando el funcionamiento de la aplicación. Este patrón permite desarrollar sistemas más organizados, mantenibles y fáciles de comprender.



¿COMO SE CONFORMA?

Vista

Es la interfaz gráfica que el usuario ve e interactúa, además de mostrar información solicitada en el programa.

Modelo

Representa los datos y la lógica de negocio de la aplicación. Gestiona el estado y las operaciones sobre la información.

Controlador

Coordina la comunicación entre Modelo y Vista. Procesa eventos del usuario y actualiza los componentes.



Patrón arquitectónico que separa la aplicación en tres componentes: Modelo, Vista y Controlador, organizando responsabilidades para mejorar el mantenimiento y la evolución del software.



MODELO (MODEL)



Representa los datos y las reglas de negocio de la aplicación. Es el núcleo que gestiona la información sin preocuparse por cómo se muestra.

Model.CourseRegisterModel

```
-courseName_M: String
-professorCourseName_M: String
-courseCode_M: String

+setCourseName(courseName_M: String): void
+getCourseName(): String
+setCourseCode(courseCode_M: String): void
+getCourseCode(): String
+setprofessorCourseName(professorCourseName_M: String): void
+getprofessorCourseName(): String
```

01

Ejemplos Concretos

Usuarios, productos, reservas, cálculos matemáticos y validaciones de datos del sistema.

02

Solo Datos y Lógica

El Modelo no conoce la interfaz visual. Procesa información independientemente de cómo se presente.

03

Conexión a Base de Datos

Gestiona la persistencia: guardar, consultar, actualizar y eliminar registros en la base de datos.

04

Relación con POO

Clases Java que encapsulan estado (atributos) y comportamiento (métodos) siguiendo principios de encapsulamiento.

+

VISTA (VIEW)

La Vista es lo que el usuario ve e interactúa: ventanas, botones, tablas, formularios e imágenes además de mostrar información del Modelo. En POO, son objetos visuales que representan el estado del modelo.

+

View.CourseRegisterView

```
panel: JPanel
+courseNameTXT: JTextField
+courseCodeTXT: JTextField
+professorCourseNameTXT: JTextField
+saveCourseButton: JButton
+cleanTextFieldsButton: JButton
```

```
+CourseRegisterView()
-createWindow(): void
-createPanel(): void
-createLabels(): void
-createTextFields(): void
-createButtons(): void
```

+

CONTROLADOR (CONTROLLER)

El Controlador actúa como intermediario entre el Modelo y la Vista. Escucha eventos del usuario, procesa acciones y coordina la actualización de ambos paquetes (View, Controller). Ejemplos: botón presionado, selección de datos, envío de formularios. En POO representa la clase coordinadora.

+

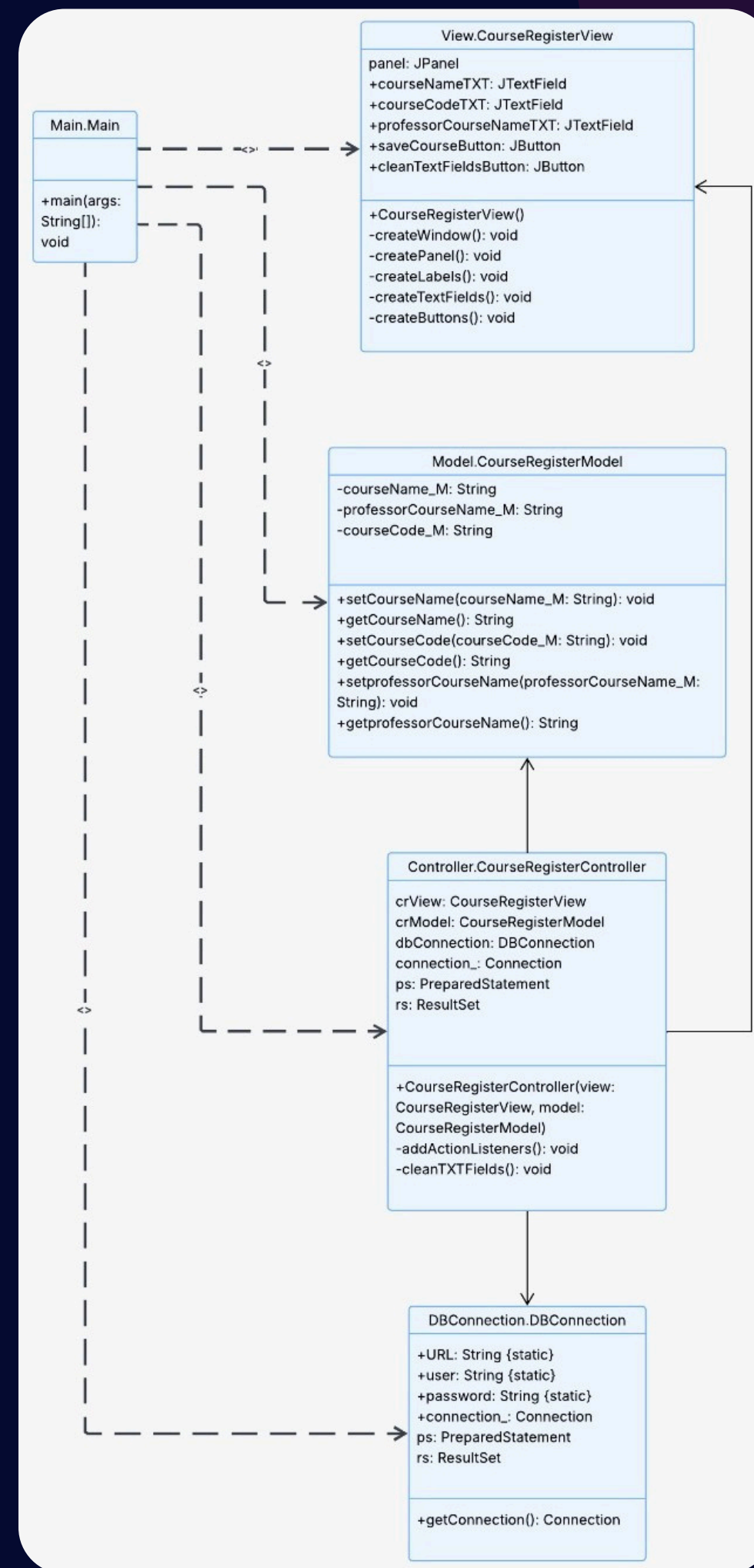
Controller.CourseRegisterController

```
crView: CourseRegisterView  
crModel: CourseRegisterModel  
dbConnection: DBConnection  
connection_: Connection  
ps: PreparedStatement  
rs: ResultSet
```

```
+CourseRegisterController(view:  
CourseRegisterView, model:  
CourseRegisterModel)  
-addActionListeners(): void  
-cleanTXTFields(): void
```

Diagrama completo

En la imagen se puede apreciar como funcionan las relaciones entre el patrón MVC en un ejemplo real



PROCESO DE INTERACCIÓN MVC

+

El flujo MVC sigue una secuencia clara: el usuario interactúa con la Vista, que notifica al Controlador, quien coordina con el Modelo para procesar datos y actualizar la interfaz.

01

Usuario interactúa con Vista

El usuario realiza una acción en la interfaz gráfica, como presionar el botón "Guardar" o ingresar datos.

02

Vista notifica al Controlador

La Vista captura el evento y lo envía al Controlador correspondiente para su procesamiento.

03

Controlador procesa y Modelo actúa

El Controlador valida la acción y coordina con el Modelo para guardar, consultar o calcular datos.

04

Vista se actualiza

La Vista refleja los cambios, ejemplo: muestra mensaje de éxito, actualiza tabla o presenta error al usuario (BD).



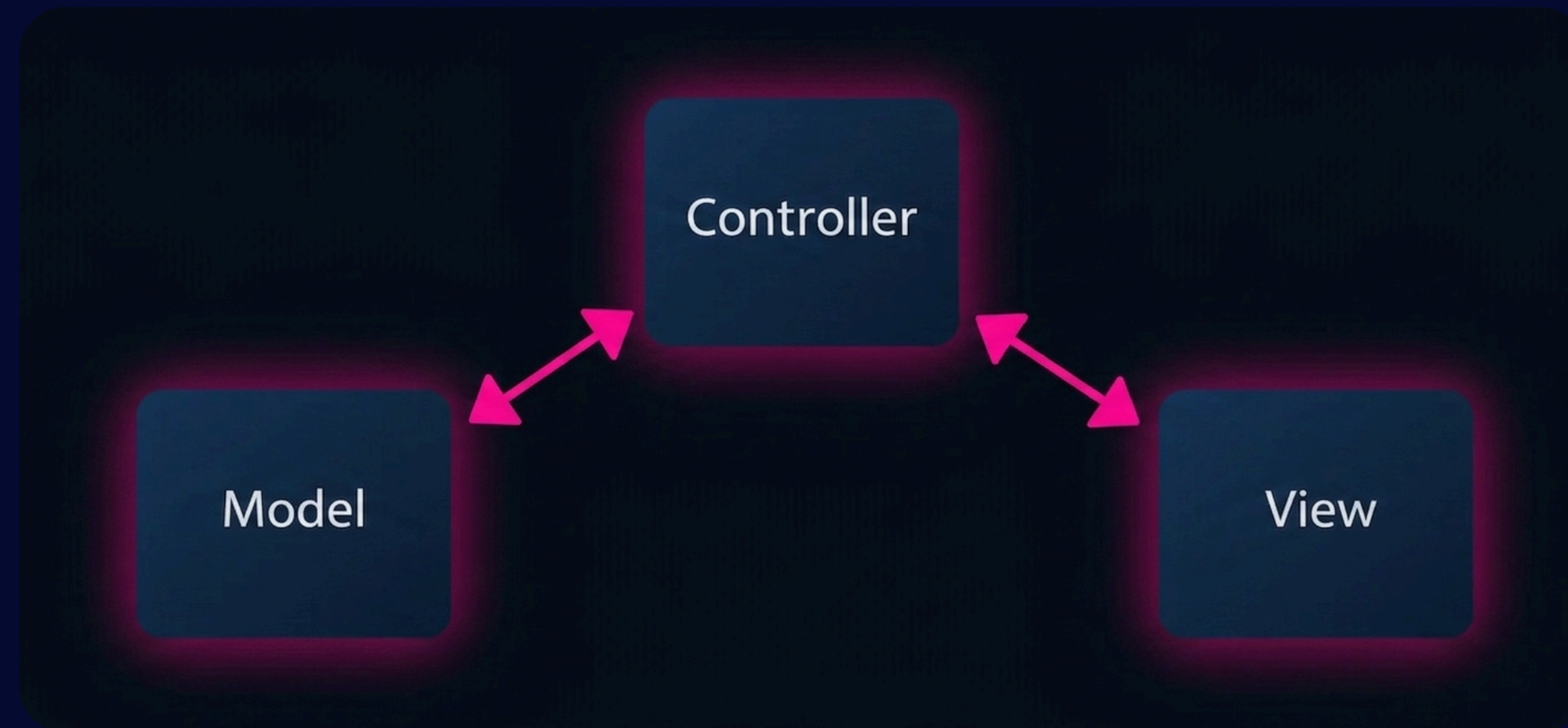


DIAGRAMA RELACIONAL MVC

Visualización del flujo de datos y comunicación entre los tres componentes principales del patrón Modelo-Vista-Controlador.

Vista

Presenta la información al usuario y captura sus interacciones para enviarlas al Controlador.

Controlador

Procesa solicitudes, coordina el flujo de información y actualiza tanto el Modelo como la Vista.

Modelo

Gestiona los datos, respondiendo a las solicitudes del Controlador, ya sea para recibir o exportar.



IMPORTANCIA Y BENEFICIOS

El patrón MVC ofrece ventajas clave que mejoran la calidad del software y facilitan el desarrollo colaborativo en proyectos de cualquier escala.

01

Organización Clara

Responsabilidades definidas para cada componente, facilitando la comprensión y navegación del código.

02

Facilita Mantenimiento

Permite modificar proyecto sin mucho esfuerzo en la búsqueda de puntos específicos, además de reducir errores y tiempo en el desarrollo.

03

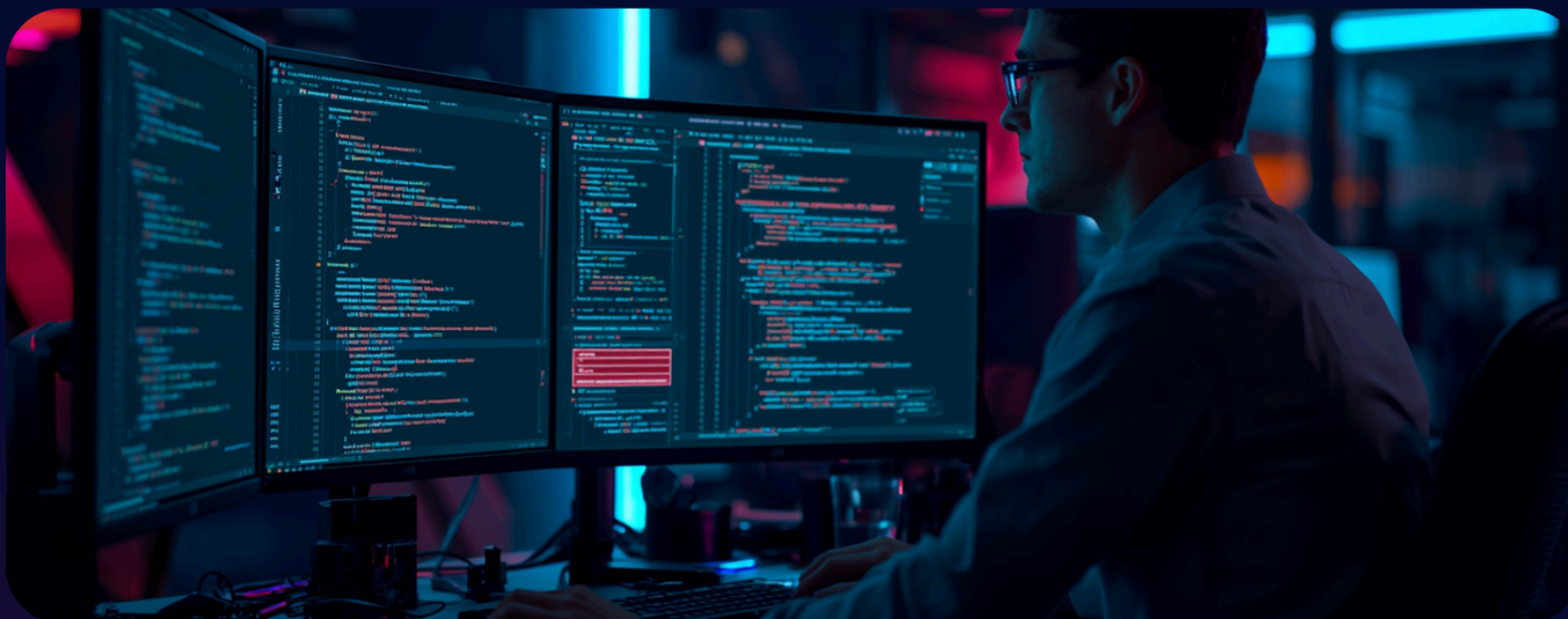
Trabajo en Equipo

Roles especializados permiten que diseñadores y programadores trabajen en paralelo sin conflictos.

04

Reutilización

El mismo modelo puede usarse en múltiples plataformas: web, móvil y escritorio con diferentes vistas.





EJEMPLO PRÁCTICO Y POO

Las clases Java representan el Modelo (datos y lógica), los archivos FXML definen la Vista (diseño visual), y las clases Controller gestionan la interacción. Esta separación aplica principios de POO: encapsulamiento, modularidad y responsabilidad única.



EQUIVALENCIAS EN JAVA Y FXML

Modelo = Clases Java

Clases Java que encapsulan datos y reglas de negocio: entidades, servicios y conexiones a base de datos.

Vista = FXML

Archivo FXML que define la estructura visual de la interfaz: botones, campos de texto, tablas y layouts declarativos.

Controlador = Controller

Clase Controller que escucha eventos del usuario, coordina la lógica y actualiza tanto el Modelo como la Vista.



Esta correspondencia permite separar responsabilidades: diseño visual en FXML, lógica de datos en clases Java, y coordinación en el Controller.





FUENTES CONSULTADAS

Referencias y materiales utilizados para la elaboración de esta presentación sobre el patrón Modelo-Vista-Controlador.

01

Documentación Oficial

Oracle Java Documentation, JavaFX API Reference y guías oficiales de desarrollo.

02

Libros de Arquitectura

"Design Patterns" de Gang of Four, "Clean Architecture" de Robert C. Martin.

03

Artículos Especializados

Publicaciones académicas y técnicas sobre patrones de diseño y arquitectura MVC.

04

Recursos Web

Tutoriales de Baeldung, Stack Overflow, GeeksforGeeks y documentación de la comunidad Java.



+

¡GRACIAS POR SU ATENCIÓN!

+

¿Preguntas? Esperamos que esta presentación haya sido útil para comprender el patrón MVC y su aplicación en interfaces gráficas organizadas y mantenibles.